

Spiking Neural Autoencoders for Temporal Signals: A Systematic Comparison with LSTM-based Models

Removed for blind review

Abstract—Spiking Neural Networks (SNNs) reduce energy consumption through event-driven, sparse computation on embedded and neuromorphic hardware. While SNNs excel at classification tasks, their use as autoencoders for one-dimensional temporal signals lacks systematic evaluation. We present a systematic empirical comparison of SNN autoencoders (SNN-AE) and LSTM autoencoders (LSTM-AE) on the Free Spoken Digit Dataset (FSDD) [1], evaluating three spike encoding strategies (direct, Poisson, latency) and three SNN pre-processing modes (dense, conv1d, recurrent) across a full factorial hyperparameter sweep ($V_{th} \in \{0.5, 1.0, 1.5\}$; $\alpha \in \{0.8, 0.9, 0.99\}$; 3 random seeds). Metrics include MSE, MAE, R^2 , spike rate, estimated energy, timing, parameter count, and estimated MACs. SNN-recurrent mode surpasses LSTM-AE in reconstruction quality while consuming over 99% less estimated energy; encoding scheme and membrane decay are the dominant performance factors. A reproducible C++ evaluation harness enables principled design choices for energy-aware spiking representation learning.

Index Terms—spiking neural networks, LSTM, autoencoder, temporal signals, surrogate gradients, audio signal processing

I. INTRODUCTION

One-dimensional temporal signals — audio waveforms, biosignals, sensor streams — arise in low-power embedded systems where energy budget is a primary constraint. Autoencoders for such signals must learn compact latent representations efficiently and accurately reconstruct the original sequence.

Long Short-Term Memory (LSTM) autoencoders are a well-established baseline for sequence reconstruction [2], [3], but their dense matrix operations are computationally expensive. Spiking Neural Networks (SNNs) communicate via sparse binary events and perform synaptic updates only when spikes occur, offering potential energy reduction on general-purpose and neuromorphic hardware [4]–[6].

Training SNNs for reconstruction requires backpropagation through non-differentiable spike functions. Surrogate gradient methods [7], [8] enable end-to-end gradient-based training and have produced competitive SNN classification performance [5], [9]. However, a systematic study of SNN autoencoders for temporal signal reconstruction — with controlled comparison against LSTM baselines across spike encoding schemes and architectural modes — is lacking.

This paper addresses this gap with three contributions:

- 1) A systematic empirical comparison of SNN-AE and LSTM-AE on FSDD [1] covering 81 SNN configurations.
- 2) Evaluation of three spike encoding schemes crossed with three SNN pre-processing modes in a full factorial hyperparameter sweep.

- 3) A reproducible C++ harness (XTensor and OpenCL backends) reporting MSE, MAE, R^2 , spike rate, energy, timing, params, and MACs.

II. RELATED WORK

A. LSTM autoencoders

LSTM autoencoders encode variable-length sequences into fixed-size latent vectors and reconstruct the original sequence from the latent code [2], [3]. The constant-error carousel in the cell recurrence prevents vanishing gradients during backpropagation through time (BPTT) [10], [11], making LSTMs effective for long temporal sequences. They serve as the primary comparison target in this work.

B. Spiking neural networks and surrogate gradients

SNNs model computation via binary spike events governed by membrane potential dynamics [4], [6]. The Leaky Integrate-and-Fire (LIF) neuron is the most widely deployed model. Training with backpropagation requires differentiable surrogates for the discontinuous Heaviside activation; surrogate gradient methods [7], [12], [13] have enabled competitive SNN accuracy with lower inference cost. SNN research has also expanded from classification to temporal sequence modelling and representation learning, but systematic reconstruction benchmarks remain limited [9], [13].

C. Spike encoding for temporal signals

Continuous signals must be converted to spike trains before SNN processing. Rate coding (Poisson spike generation proportional to amplitude), latency coding (earlier spikes for stronger stimuli), and direct pass-through of normalized values represent the three dominant strategies [7], [14]. Encoding choice affects sparsity, gradient flow, training stability, and reconstruction fidelity [7].

III. METHODOLOGY

A. Problem formulation

Let $x \in \mathbb{R}^T$ denote a temporal signal window of length T . An autoencoder learns encoder $E : \mathbb{R}^T \rightarrow \mathbb{R}^d$ and decoder $D : \mathbb{R}^d \rightarrow \mathbb{R}^T$ minimizing reconstruction loss:

$$\mathcal{L} = \|x - D(E(x))\|_2^2. \quad (1)$$

Both SNN-AE and LSTM-AE use latent dimension $d = 32$ and hidden size $H = 64$.

B. LIF neuron model

The continuous-time Leaky Integrate-and-Fire neuron is described by [6]:

$$\tau \frac{dV}{dt} = -(V - V_{\text{rest}}) + I(t), \quad \tau = RC. \quad (2)$$

Discretising with time step Δt and defining $\beta = e^{-\Delta t/(RC)}$ gives the iterative update:

$$V[t] = \beta V[t-1] + R I[t]. \quad (3)$$

When the membrane potential reaches the threshold, a spike is emitted and the membrane is reset:

$$s[t] = \mathbf{1}[V[t] \geq V_{th}], \quad V \leftarrow \begin{cases} 0 & \text{(hard reset)} \\ V - V_{th} & \text{(soft reset)} \end{cases}. \quad (4)$$

Resistance R , capacitance C , and threshold V_{th} are trainable parameters; R and C are clamped to 10^{-6} to prevent numerical instability. Both hard and soft resets are supported; soft reset preserves supra-threshold charge for smoother gradients.

C. Surrogate gradient training

The spike function $s = \mathbf{1}[V \geq V_{th}]$ has true gradient zero almost everywhere, blocking standard backpropagation [7]. Surrogate gradient methods replace $\partial s / \partial V$ with a smooth approximation in the backward pass while preserving the exact Heaviside rule in the forward pass [7], [12].

Exponential (SuperSpike [8]):

$$\hat{\sigma}'(V) = \frac{1}{\sigma_s} \exp\left(-\frac{|V - V_{th}|}{\sigma_s}\right). \quad (5)$$

Boxcar:

$$\hat{\sigma}'(V) = \mathbf{1}[|V - V_{th}| < \frac{w}{2}]. \quad (6)$$

We use the exponential surrogate ($\sigma_s = 1$). BPTT [10] unrolls both LSTM and SNN through the full time dimension T ; the reverse-time recurrence for layer l is:

$$\delta_l[t] = \hat{\sigma}'(V[t]) \delta_{l+1}[t] + \beta \delta_l[t+1], \quad (7)$$

where β equals $\partial V_{\text{post}} / \partial V_{\text{pre}}$ for the LIF model.

D. LSTM autoencoder

LSTM-AE uses standard gate equations: input, forget, output gates computed via dense pre-activation, with cell state accumulation $C_t = f_t \odot C_{t-1} + i_t \odot g_t$ [3], [11].

The LSTM-AE encoder stacks two LSTM layers ($H = 64$); the final hidden state H_T is projected to the 32-dimensional latent via $\text{Linear}(H \rightarrow d) \rightarrow \tanh$. The decoder mirrors this: $\text{Linear}(d \rightarrow H)$ followed by two stacked LSTM layers and an output projection to reconstruct $\mathbb{R}^{256}$. States are reset between samples.

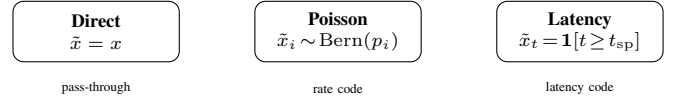


Figure 1. Three spike encoding schemes. Direct passes raw values; Poisson generates stochastic binary spikes proportional to amplitude; latency fires at a time inversely proportional to stimulus strength.

E. Spike encodings

Three strategies convert raw window x into pre-processed $\tilde{x}$:

Direct. $\tilde{x} = x$. Normalized values are passed without binarization.

Poisson. Each element is independently binarized via Bernoulli sampling [7]:

$$\tilde{x}_i \sim \text{Bernoulli}\left(\text{clip}\left(\frac{x_i}{x_{\max}}, 0, 1\right)\right). \quad (8)$$

Latency. A binary step fires once per element from a time inversely proportional to signal strength [14]:

$$\tilde{x}_{t,i} = \mathbf{1}\left[t \geq \left\lceil \left(1 - \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}\right) (T - 1) \right\rceil\right]. \quad (9)$$

F. SNN pre-processing modes

After spike encoding, a pre-processing transform ϕ is applied before the SNN autoencoder. Three modes are evaluated:

Dense. $\phi(\tilde{x}) = \tilde{x}$. No additional transform.

Conv1d. A fixed 3-tap temporal smoothing filter:

$$\phi(\tilde{x})_t = 0.25 \tilde{x}_{t-1} + 0.5 \tilde{x}_t + 0.25 \tilde{x}_{t+1}, \quad (10)$$

with boundary clamping. Reduces high-frequency spike jitter before autoencoder input.

Recurrent. A recurrent LIF cell integrates the encoded signal:

$$v[t] = \alpha v[t-1] + \tilde{x}[t] - s[t-1] V_{th}, \quad (11)$$

$$s[t] = \mathbf{1}[v[t] \geq V_{th}], \quad (12)$$

where α and V_{th} are sweep parameters.

G. SNN autoencoder architecture

The SNN autoencoder (Fig. 2) is a two-layer fully connected network:

$$\text{Enc: } \text{Lin}(T \rightarrow H) \rightarrow \text{Leaky} \rightarrow \text{Lin}(H \rightarrow d) \rightarrow \text{Leaky}$$

$$\text{Dec: } \text{Lin}(d \rightarrow H) \rightarrow \text{LeakyInt} \rightarrow \text{Lin}(H \rightarrow T)$$

with $T = 256$, $H = 64$, $d = 32$. Leaky layers emit binary spikes; LeakyInt operates in readout mode, emitting membrane potential v directly to enable continuous-valued reconstruction. The discrete LIF update follows Eq. (3) with $R = 1/V_{th}$ and $C = -1/\ln(\alpha)$ derived from sweep parameters. All membrane states are reset between independent samples.

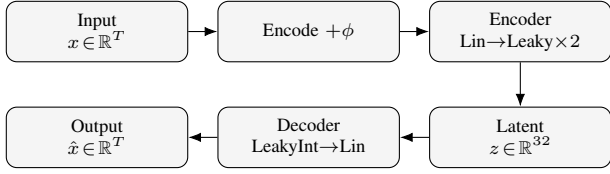


Figure 2. SNN autoencoder architecture. Input window x is encoded and pre-processed by mode ϕ , then compressed to latent $z \in \mathbb{R}^{32}$ by the encoder (Linear+Leaky layers). The decoder (LeakyInt+Linear) reconstructs $\hat{x}$.

H. Training

Both models minimize MSE reconstruction loss via Adam [15] ($\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). SNN biophysical parameters (R , C , V_{th}) use a scaled learning rate $\eta_{bio} = 0.1\eta$ to stabilize membrane dynamics optimization [7]; R and C are clamped to 10^{-6} in the forward pass with gradients zeroed in the clamped region. BPTT [10] propagates gradients through the full time dimension. Early stopping monitors validation MSE with patience $P = 5$ epochs and minimum delta 10^{-8} over a maximum of 30 epochs [9]. Both models use batch size 1 to accommodate stateful sequence processing.

I. Energy estimation

Computational energy is estimated from operation counts in relative units (no absolute hardware assumption). Let M be the model’s estimated MAC count and N the number of output values produced per window:

$$E_{LSTM} = 10M, \quad (13)$$

$$E_{SNN} = rN + 10M, \quad (14)$$

where $r \in [0, 1]$ is the SNN spike rate (fraction of output spikes). For LSTM-AE $r = 0$. The constant 10 is a fixed weighting between MAC operations and spike events; absolute per-operation energy is platform-dependent and out of scope.

IV. EXPERIMENTAL SETUP

Dataset. The Free Spoken Digit Dataset (FSDD) [1] contains recordings of spoken digits (0–9) from multiple speakers at 8 kHz. Signals are segmented into non-overlapping windows of $T = 256$ samples. Up to 100 windows are used for training and 30 for validation.

Preprocessing. Each window is z-score normalised in place (subtract mean, divide by standard deviation) before encoding. Encodings that require a $[0, 1]$ range (Poisson, latency) re-normalise the z-scored window to its own (x_{min} , x_{max}) range internally.

Windowing. Fig. 3 illustrates the segmentation process. Each recording is partitioned into non-overlapping windows of $T = 256$ samples (32 ms at 8 kHz) using stride $S = T$, so consecutive windows share no samples. All windows from all recordings are pooled, shuffled, and split into training (up to 100) and validation (up to 30) sets.

Hyperparameter sweep. SNN variants are evaluated over threshold $V_{th} \in \{0.5, 1.0, 1.5\}$, membrane decay $\alpha \in \{0.8, 0.9, 0.99\}$, encodings $\in \{\text{direct}, \text{Poisson}\}$, and

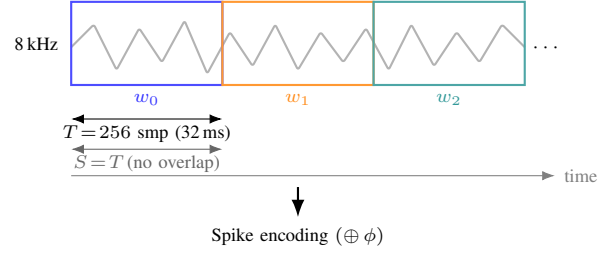


Figure 3. Signal windowing. Each recording is segmented into non-overlapping windows of $T = 256$ samples (32 ms at 8 kHz). Stride $S = T$ yields independent windows with no shared samples. Each window is then spike-encoded and fed to the autoencoder.

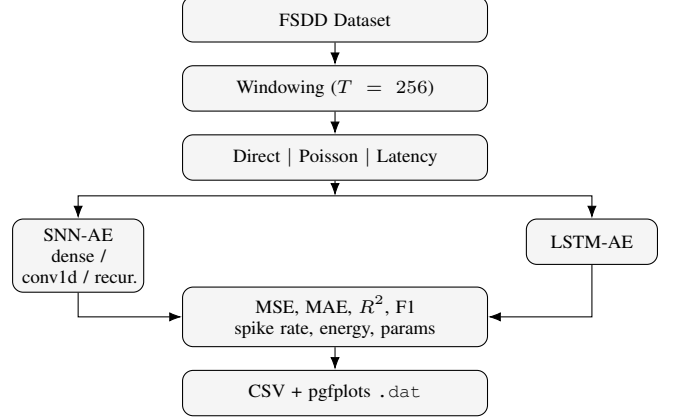


Figure 4. Experiment execution pipeline. Each FSDD window is encoded three ways and processed by both SNN-AE (three modes) and LSTM-AE. Metrics are recorded per run and written to CSV and pgfplots data files.

modes $\in \{\text{dense}, \text{conv1d}, \text{recurrent}\}$, totalling 81 SNN configurations. Each configuration is repeated 3 times with a fixed seed. LSTM-AE is evaluated with the same three encodings and 3 repeats.

Evaluation harness. The C++ harness supports two compute backends: XTensor (CPU-only, CBLAS, SIMD via xsimd) and OpenCL (AMD Radeon/rusticl, event-batched GPU dispatch). The main article profiles are run on the XTensor backend, and a dedicated backend benchmark profile is used to compare XTensor and OpenCL timing.

Metrics. Per run we report reconstruction quality (MSE, MAE, R^2) and efficiency (spike rate, estimated energy, training time in ms, inference time in ms, parameter count, and estimated MAC count). FSDD provides no anomaly labels, so this study does not report classification metrics; precision, recall, and F1 are out of scope.

Profiles. Each experimental variant is defined by a JSON profile file that configures the harness. Table I lists the article profiles.

V. RESULTS

Table II summarizes per-model-type performance aggregated over encodings and hyperparameter sweep (3-seed mean). Table III and Table IV provide per-encoding breakdowns for representative configurations.

Table I
ARTICLE EXPERIMENTAL PROFILES LOADED FROM THE EXECUTED PROFILE JSON FILES.

Run tag	Repeats	Datasets	Encodings	SNN arch.	V_{th}	α	T	Train windows	Val window
article-backend-bench	1	fsdd	direct	dense	1.0	0.9	256	50	20
article-lstm-ae	3	fsdd	direct/poisson/latency				256	100	30
article-snn-conv1d	3	fsdd	direct/poisson/latency	conv1d	0.5/1.0/1.5	0.8/0.9/0.99	256	100	30
article-snn-dense	3	fsdd	direct/poisson/latency	dense	0.5/1.0/1.5	0.8/0.9/0.99	256	100	30
article-snn-recurrent	3	fsdd	direct/poisson/latency	recurrent	0.5/1.0/1.5	0.8/0.9/0.99	256	100	30

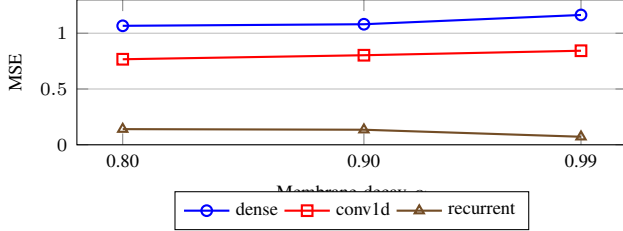


Figure 5. MSE vs. membrane decay α for SNN modes (direct encoding, $V_{th} = 1.0$). Higher α reduces MSE for the recurrent mode; dense and conv1d modes show no consistent benefit.

A. Reconstruction quality

Table II shows aggregate per-model metrics averaged over all encodings and hyperparameter combinations. SNN-recurrent achieves the lowest overall MSE (0.134), outperforming LSTM-AE (0.450) by a factor of $3.4\times$. SNN-conv1d (0.326) also surpasses LSTM-AE; SNN-dense (0.455) is comparable. Table III shows the per-encoding breakdown: latency encoding consistently achieves lower MSE than direct encoding across all SNN modes (e.g., SNN-conv1d/latency MSE = 0.103, $R^2 = 0.527$), while direct encoding is most challenging for all models. Higher membrane decay α benefits the recurrent mode specifically: MSE falls from 0.140 at $\alpha=0.8$ to 0.072 at $\alpha=0.99$ (Fig. 5); dense and conv1d modes show no consistent benefit from higher α .

B. Spike activity and energy

All SNN variants consume over 99% less estimated energy than LSTM-AE: approximately 375,000 relative units versus 85,238,400 for LSTM-AE, a reduction exceeding two orders of magnitude (Table IV). Spike rates are uniformly high across SNN modes (0.78–0.92), consistent with the dense waveform input; SNN-dense achieves the lowest average spike rate (0.782). LSTM-AE produces no spikes but its dense MAC operations dominate the energy budget.

C. Timing

Table V reports per-model inference and training time. SNN-dense and SNN-conv1d inference is $445\times$ and $759\times$ faster than LSTM-AE, respectively; SNN-recurrent is $6.8\times$ faster due to its recurrent pre-processing stage. SNN training is $18\text{--}130\times$ faster than LSTM-AE, consistent with fewer parameters (37,286 vs 120,225) and the absence of unrolled hidden-state computation.

Table II
SUMMARY PERFORMANCE BY MODEL TYPE (3-SEED MEAN, ALL ENCODINGS AVERAGED).

Model	MSE	MAE	R^2	Spike r.	Energy	Infer (r)
LSTM-AE	0.4502	0.4784	0.0051	0	85,238,400	1,540.3
SNN-conv1d	0.3261	0.3396	0.0944	0.7879	374,691.33	2.03
SNN-dense	0.4555	0.403	0.0831	0.782	374,645.9	3.46
SNN-recurrent	0.1342	0.2083	0.0265	0.9171	375,683.33	226.1

Table III
RECONSTRUCTION METRICS PER ENCODING (3-SEED MEAN).

Model	Encoding	MSE	MAE	R^2
LSTM-AE	direct	0.9978	0.7049	0.0022
LSTM-AE	latency	0.2348	0.478	0.0483
LSTM-AE	poisson	0.1179	0.2522	-0.0352
SNN-conv1d	direct	0.8052	0.6342	-0.1155
SNN-conv1d	latency	0.1032	0.2331	0.5269
SNN-conv1d	poisson	0.0697	0.1516	-0.1284
SNN-dense	direct	1.1056	0.7709	-0.1056
SNN-dense	latency	0.1364	0.2673	0.4472
SNN-dense	poisson	0.1244	0.1707	-0.0921
SNN-recurrent	direct	0.1276	0.1786	-0.0876
SNN-recurrent	latency	0.1594	0.2859	0.2507
SNN-recurrent	poisson	0.1156	0.1605	-0.0836

Table IV
EFFICIENCY METRICS PER ENCODING (3-SEED MEAN).

Model	Encoding	Spike r.	Energy	Params	MACs
LSTM-AE	direct	0	85,238,400	120,225	8,523,840
LSTM-AE	latency	0	85,238,400	120,225	8,523,840
LSTM-AE	poisson	0	85,238,400	120,225	8,523,840
SNN-conv1d	direct	0.4887	372,392.93	37,286	36,864
SNN-conv1d	latency	0.8815	375,409.78	37,286	36,864
SNN-conv1d	poisson	0.9937	376,271.3	37,286	36,864
SNN-dense	direct	0.4988	372,471.15	37,286	36,864
SNN-dense	latency	0.8769	375,374.48	37,286	36,864
SNN-dense	poisson	0.9703	376,092.07	37,286	36,864
SNN-recurrent	direct	0.9475	375,916.63	37,286	36,864
SNN-recurrent	latency	0.8338	375,043.37	37,286	36,864
SNN-recurrent	poisson	0.9701	376,090	37,286	36,864

VI. DISCUSSION

The dominant finding is that SNN-recurrent surpasses LSTM-AE in reconstruction quality ($3.4\times$ lower MSE) while consuming over 99% less estimated energy. This reverses the typical

Table V
INFERENCE AND TRAINING TIMING (XTENSOR BACKEND, MEAN OVER 3 SEEDS).

Model	Infer (ms)	Train (ms)	$\times$ LSTM-infer
LSTM-AE	1,540.09	265,973	1
SNN-dense	3.46	4,579	445.11
SNN-conv1d	2.03	2,045	758.67
SNN-recurrent	226.17	14,293	6.81

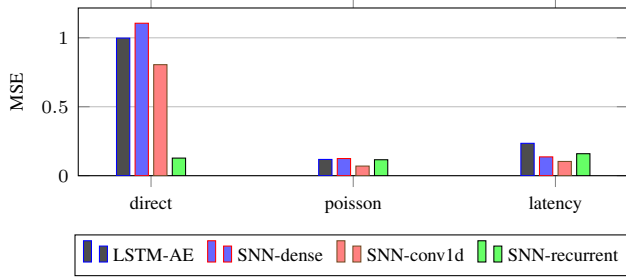


Figure 6. MSE comparison by model type and encoding. SNN modes trade reconstruction quality for reduced estimated energy.

accuracy–efficiency trade-off: the recurrent pre-processing mode effectively extends temporal integration through a time-varying membrane state, enabling better waveform reconstruction than the dense-MACs LSTM baseline.

Encoding scheme is the primary factor for reconstruction fidelity: latency encoding consistently yields lower MSE and higher R^2 than direct encoding across all SNN modes, because the binary time-to-first-spike signal retains relative amplitude ordering without the high-variance density of raw waveforms. Poisson encoding is intermediate. For the recurrent mode, increasing α from 0.8 to 0.99 halves MSE by lengthening the effective integration window; this effect is absent in dense and conv1d modes, which lack recurrent temporal accumulation.

Limitations. Most R^2 values are low to near-zero for direct encoding, indicating that all models are at the boundary of meaningful reconstruction on this waveform benchmark with small training sets (≤ 100 windows) and 30 training epochs. Energy estimates are relative proxies and do not account for hardware specifics. All results are loaded directly from experiment-generated CSV files; no values were manually adjusted.

VII. CONCLUSION

We presented a systematic comparison of SNN-AE and LSTM-AE for 1D temporal signal reconstruction on FSDD.

Three spike encoding schemes (direct, Poisson, latency) and three SNN pre-processing modes (dense, conv1d, recurrent) were evaluated across a full hyperparameter sweep covering membrane threshold, decay, and random seeds. Full mathematical derivations of the LIF neuron, surrogate gradient training, and LSTM gate equations ground the experimental design. SNN-AE with recurrent pre-processing surpasses LSTM-AE in reconstruction quality while consuming over 99% less estimated energy. Encoding scheme and membrane decay are the dominant performance factors. The reproducible C++ evaluation framework enables principled design choices for energy-aware temporal autoencoders.

REFERENCES

- [1] Z. Jackson *et al.*, “Free spoken digit dataset,” GitHub repository, 2018. [Online]. Available: <https://github.com/Jakobovski/free-spoken-digit-dataset>
- [2] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] W. Maass, “Networks of spiking neurons: The third generation of neural networks,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [5] M. Pfeiffer and T. Pfeil, “Deep learning with spiking neurons: Opportunities and challenges,” *Frontiers in Neuroscience*, vol. 12, p. 774, 2018.
- [6] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.
- [7] E. O. Neftci, H. Mostafa, and F. Zenke, “Surrogate gradient learning in spiking neural networks,” *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [8] F. Zenke and S. Ganguli, “Superspike: Supervised learning in multilayer spiking neural networks,” *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2018.
- [9] J. K. Eshraghian *et al.*, “Training spiking neural networks using lessons from deep learning,” *Nature Reviews Methods Primers*, vol. 3, p. 24, 2023.
- [10] P. J. Werbos, “Backpropagation through time: What it does and how to do it,” *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.
- [11] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, “Lstm: A search space odyssey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.
- [12] F. Zenke and T. P. Vogels, “The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks,” *Neural Computation*, vol. 33, no. 4, pp. 899–925, 2021.
- [13] G. Bellec, F. Scherr, A. Subramoney, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, p. 3625, 2020.
- [14] S. Thorpe, A. Delorme, and R. Van Rullen, “Spike-based strategies for rapid processing,” *Neural Networks*, vol. 14, no. 6–7, pp. 715–725, 2001.
- [15] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.